CS 4310 Operating Systems
Project #1 Simulating Job Scheduler and Performance Analysis

(Total: 100 points)

Student Name: Addison Taylor

Date: 03-04-26

Important:
- *Please read this document completely before you start coding.*
- Also, please read the submission instructions (provided at the end of this document) carefully before submitting the project.

Project #1 Description:

Simulating Job Scheduler of the Operating Systems by programming the following four scheduling algorithms that we covered in the class:

a. First-Come-First-Serve (FCFS)
a. Shortest-Job-First (SJF)
b. Round-Robin with Time Slice = 2 (RR-2)
c. Round-Robin with Time Slice = 5 (PR-5)

You can use either Java or your choice of programming language for the implementation. The objective of this project is to help student understand how above four job scheduling algorithms operates by implementing the algorithms, and conducting a performance analysis of them based on the performance measure of their average turnaround times (of all jobs) for each scheduling algorithm using multiple inputs. Output the details of each algorithm's execution. You need to show which jobs are selected at what times as well as their starting and stopping burst values. You can choose your display format, for examples, you can display the results of each in *Schedule Table* or *Gantt Chart* format (as shown in the class notes). The project will be divided into three parts (phases) to help you to accomplish above tasks in in a systematic and scientific fashion: Design and Testing, Implementation, and Performance Analysis.

The program will read process burst times from a file (job.txt) – this file will be generated by you. Note that you need to generate multiple testing cases (with inputs of 5 jobs, 10 jobs and 15 jobs). A sample input file of five jobs is given as follows (burst time in ms):

[Begin of job.txt]
Job1
7
Job2
18
Job3
10
Job4
4
Job5
12
[End of job.txt]

Note: you can assume that

(1) There are no more than 30 jobs in the input file (job.txt).
(2) Processes arrive in the order they are read from the file for FCFS, RR-2 and RR-5.
(3) All jobs arrive at time 0.
(4) FCFS uses the order of the jobs, Job1, Job2, Job3, …

You can implement the algorithms in your choice of data structures based on the program language of your choice. Note that you always try your best to give the most efficient program for each problem. The size of the input will be limited to be within 30 jobs.

Submission Instructions:

- ***turn in the following @Canvas after the completion of all three parts, part 1, part 2 and part 3***
  (1) four program files (your choice of programming language with proper documentation)
  (2) this document (complete all the answers)

Part1
Design & Testing (30 points)

a. Design the program by providing pseudocode or flowchart for each CPU scheduling algorithm.

First-Come-First-Serve (FCFS):

```
start time = 0
end time = 0
index = 0
total time = 0

while(lines to be read) {
        read in job name, store in array
        read in job length, store in array
}

while(index < array length) {                   // Perform [job name] for [job length] units of time
        output name of job at index, start time
        end time = end time + [job length]
        start time = start time + [job length]
        store end time in array
        output end time
        index = index + 1
}
for (end time array length) {                   // Add up end times for each job
        add value of end time array at index to total time
}
output (total time / number of jobs)            // Average turnaround time
```

Shortest-Job-First (SJF):

```
start time = 0
end time = 0
index = 0
total time = 0

while(lines to be read) {
        read in job name, store in array
        read in job length, store in array
}

while(index < array length) {                   // Check for shortest job and perform that
        shortest job = MAX
        for (array length) {                    // Find shortest job
                if ([job name] length < shortest job) {
                        shortest job = [job name] index
                }
        }
        output name of shortest job, start time
        end time = end time + [job length]
        start time = start time + [job length]
        store end time in array
        output end time
        set shortest job length to MAX          // So it is not performed more than once
        index = index + 1
```

```
}
for (end time array length) {                    // Add up end times for each job
        add value of end time array at index to total time
}
output (total time / number of jobs)             // Average turnaround time
```

Round-Robin with Time Slice = 2 (RR-2):

```
start time = 0
end time = 0
index = 0
total time = 0

while(lines to be read) {
        read in job name, store in array
        read in job length, store in array
}
number of jobs = array length

while(number of jobs > 0) {                      // Perform [job name] for up to 2 units of time
        output name of job at index, start time
        if ([job name] time left >= 2) {
                end time += 2
                start time += 2
                [job name] time left -= 2
                output end time

                if ([job name] time left == 0) {         // If job is complete after, decrease number of jobs left
                        store end time in array
                        number of jobs -= 1
                }
        }
        else {                                           // If < 2 units of time remaining, decrease number of jobs left
                if ([job name] time left == 1) {         // Only perform if job time left > 0
                        end time += 1
                        start time += 1
                        [job name] time left -= 1
                        output end time
                        store end time in array
                        number of jobs -= 1
                }
        }
        index = index + 1

        if (index == number of jobs) {                   // If one iteration is done, loop again
                index = 0
        }
}
for (end time array length) {                    // Add up end times for each job
        add value of end time array at index to total time
}
output (total time / number of jobs)             // Average turnaround time
```

Round-Robin with Time Slice = 5 (RR-5):

```
start time = 0
end time = 0
index = 0
total time = 0

while(lines to be read) {
        read in job name, store in array
        read in job length, store in array
}
number of jobs = array length

while(number of jobs > 0) {                     // Perform [job name] for up to 5 units of time
        output name of job at index, start time
        if ([job name] time left >= 5) {
                end time += 5
                start time += 5
                [job name] time left -= 5
                output end time

                if ([job name] time left == 0) {        // If job is complete after, decrease number of jobs left
                        store end time in array
                        number of jobs -= 1
                }
        }
        else {                                          // If < 5 units of time remaining, decrease number of jobs left
                if ([job name] time left > 0) {         // Only perform if job time left > 0
                        end time += [job name] time left
                        start time += [job name] time left
                        [job name] time left -= [job name] time left
                        output end time
                        store end time in array
                        number of jobs -= 1
                }
        }
        index = index + 1

        if (index == number of jobs) {                  // If one iteration is done, loop again
                index = 0
        }
}
for (end time array length) {                           // Add up end times for each job
        add value of end time array at index to total time
}
output (total time / number of jobs)                    // Average turnaround time
```

b. Design the program correctness testing cases. Give at least 3 testing cases to test your program, and give the expected correct **average turnaround time** (for each testing case) in order to test the correctness of each algorithm.

Testing case #	Input (table of jobs with its job# and length	Expected output for FCFS (√ if Correct after testing in Part 2)	Expected output for SJF (√ if Correct after testing in Part 2)	Expected output for RR-2 (√ if Correct after testing in Part 2)	Expected output for RR-5 (√ if Correct after testing in Part 2)
1 (5 jobs)	Job1 7 Job2 18 Job3 10 Job4 4 Job5 12	(7 + 25 + 35 + 39 + 51) / 5 = **31.4 ms** √	(4 + 11 + 21 + 33 + 51) / 5 = **24.0 ms** √	(18 + 29 + 39 + 45 + 51) / 5 = **36.4 ms** √	(19 + 26 + 36 + 48 + 51) / 5 = **36.0 ms** √
2 (10 jobs)	Job1 3 Job2 9 Job3 5 Job4 11 Job5 6 Job6 10 Job7 4 Job8 8 Job9 7 Job10 12	(3 + 12 + 17 + 28 + 34 + 44 + 48 + 56 + 63 + 75) / 10 = **38.0 ms** √	(3 + 7 + 12 + 18 + 25 + 33 + 42 + 52 + 63 + 75) / 10 = **33.0 ms** √	(21 + 33 + 42 + 46 + 62 + 63 + 66 + 70 + 73 + 75) / 10 = **55.1 ms** √	(3 + 13 + 32 + 51 + 57 + 62 + 65 + 67 + 73 + 75) / 10 = **49.8 ms** √
3 (15 jobs)	Job1 10 Job2 5 Job3 3	(10 + 15 + 18 + 30 + 38 + 49 + 63 + 72 + 87 + 94 + 98 + 111 + 117 + 135 + 152) / 15 = **72.6 ms** √	(3 + 7 + 12 + 18 + 25 + 33 + 42 + 52 + 63 + 75 + 88 + 102 + 117 + 134 + 152) / 15 = **61.533 ms** √	(35 + 51 + 62 + 80 + 90 + 99 + 107 + 114 + 124 + 125 + 137 + 140 + 145 + 151 + 152) / 15 = **107.466 ms** √	(10 + 13 + 52 + 77 + 85 + 99 + 106 + 112 + 124 + 125 + 129 + 134 + 137 + 150 + 152) / 15 = **100.33 ms** √

	Job4	12				
	Job5	8				
	Job6	11				
	Job7	14				
	Job8	9				
	Job9	15				
	Job10	7				
	Job11	4				
	Job12	13				
	Job13	6				
	Job14	18				
	Job15	17				

c. Design testing strategy for the programs. Discuss about how to generate and structure the randomly generated inputs for experimental study later in Part 3.

Hint 1: To study the performance evaluation of the four job scheduling algorithms, this project will use three different input sizes, 5 jobs, 10 jobs and 15 jobs. It is the easiest to use a random number generator for generating the inputs. Note that you need to decide the maximum value of job length (use at least 20). However, student should store each data set in various sizes and use the same data set for each job scheduling algorithm.

The performance of average Turnaround Time of each input data size (5 jobs, 10 jobs and 15 jobs) can be calculated after an experiment is conducted in 20 trail (with 20 input sets of jobs). We can denote the results as the set X which contains the 20 computed Turnaround Times of 20 trails, where $X = \{x_1, x_2, x_3 \ldots x_{20}\}$*, from the simulator.*

For each data size (5 jobs, 10 jobs and 15 jobs):

$$\textit{Average Turnaround Time} = \frac{\sum_{i=1}^{20} Xi}{20}$$

The student should decide the maximum value of the job length (at least 20).

Single Trial (Random):

1. Allow user to select input size *n*: 5 jobs, 10 jobs, or 15 jobs
2. Input file:
   - Write the name of the job in the input file, starting with Job1
   - Randomly generate the length of each job (between 2-25) and add to input file
   - Repeat for all *n* jobs
3. Run all 4 programs using this input file
4. Output average turnaround time for each program

Single Trial (User File):

1. Allow user to upload their own job.txt file
2. Ask user for the number of jobs in the file
3. Run all 4 programs using this input file
4. Output average turnaround time for each program

20 Trials (Random):

1. Allow user to select input size: 5 jobs, 10 jobs, or 15 jobs
2. For each of the 20 trials:
   a. Generate input file (same as from random single trial)
   b. Run all 4 programs using this file
   c. Compute and add the average turnaround time from each program to a set (one set per program)
   d. Repeat for all remaining trials
3. Compute the overall average turnaround time for each program using the information in the set and the Average Turnaround Time formula

**** Calculation difference:** I added the total time for each trial and divided by (number of jobs * 20 trials) rather than adding the average for each trial and dividing by 20 trials (for each program). I hoped to reduce rounding errors in doing so.

Part 2
Implementation (30 points)

a. Code each program based on the design (pseudocode or flow chart) in Part 1(a).

<Generate four programs and stored them in four files, needed to be submitted> √

b. Document the program appropriately.

<Generate documentation inside the four program files> √

c. Test your program using the designed testing input data given in the table in Part 1(b), Make sure each program generates the correct answer by marking a "√" if it is correct for each testing case for each program column in the table. Repeat the process of debugging if necessary.

<Complete the four columns of the four algorithms in the table @Part 1(b)> √

d. For each program, capture a screen shot of the execution (Compile&Run) using the testing case in Part 1(b) to show how this program works properly

Testing with 5 jobs:

1. First Come First Served:

```
-----------------------------------
Job:    | Length: | Start: | End:
-----------------------------------
Job1    |  7      |  0     |  7
-----------------------------------
Job2    |  18     |  7     |  25
-----------------------------------
Job3    |  10     |  25    |  35
-----------------------------------
Job4    |  4      |  35    |  39
-----------------------------------
Job5    |  12     |  39    |  51
-----------------------------------

Average Turnaround Time: (7 + 25 + 35 + 39 + 51) / 5 = 31.4 ms
```

2. Shortest Job First:

```
-----------------------------------
Job:    | Length: | Start: | End:
-----------------------------------
Job4    |  4      |  0     |  4
-----------------------------------
Job1    |  7      |  4     |  11
-----------------------------------
Job3    |  10     |  11    |  21
-----------------------------------
Job5    |  12     |  21    |  33
-----------------------------------
Job2    |  18     |  33    |  51
-----------------------------------

Average Turnaround Time: (11 + 51 + 21 + 4 + 33) / 5 = 24.0 ms
```

3. Round Robin (2):

```
Time:    | Job:     | Remaining:
0–2      |  Job1    |  5
2–4      |  Job2    |  16
4–6      |  Job3    |  8
6–8      |  Job4    |  2
8–10     |  Job5    |  10
10–12    |  Job1    |  3
12–14    |  Job2    |  14
14–16    |  Job3    |  6
16–18    |  Job4    |  0 [Complete]
18–20    |  Job5    |  8
20–22    |  Job1    |  1
22–24    |  Job2    |  12
24–26    |  Job3    |  4
26–28    |  Job5    |  6
28–29    |  Job1    |  0 [Complete]
29–31    |  Job2    |  10
31–33    |  Job3    |  2
33–35    |  Job5    |  4
35–37    |  Job2    |  8
37–39    |  Job3    |  0 [Complete]
39–41    |  Job5    |  2
41–43    |  Job2    |  6
43–45    |  Job5    |  0 [Complete]
45–47    |  Job2    |  4
47–49    |  Job2    |  2
49–51    |  Job2    |  0 [Complete]

Average Turnaround Time: (29 + 51 + 39 + 18 + 45) / 5 = 36.4 ms
```

4. Round Robin (5):

```
Time:    | Job:     | Remaining:
0–5      |  Job1    |  2
5–10     |  Job2    |  13
10–15    |  Job3    |  5
15–19    |  Job4    |  0 [Complete]
16–24    |  Job5    |  7
21–26    |  Job1    |  0 [Complete]
22–31    |  Job2    |  8
27–36    |  Job3    |  0 [Complete]
32–41    |  Job5    |  2
37–46    |  Job2    |  3
42–48    |  Job5    |  0 [Complete]
43–51    |  Job2    |  0 [Complete]

Average Turnaround Time: (26 + 51 + 36 + 19 + 48) / 5 = 36.0 ms
```

By now, four working programs are created and ready for experimental study in the next part, Part 3.

Part 3
Performance Analysis (40 points)

a. Run each program with the designed randomly generated input data given in Part 1(c). Generate a table for all the experimental results for performance analysis as follows.

Input Size n jobs	Average of average turnaround times (FCFS Program)	Average of average turnaround times (SFJ Program)	Average of average turnaround times (RR-2)	Average of average turnaround times (RR-5)
5 jobs	4289.0 ms / 100 jobs = **42.89 ms**	3446.0 ms / 100 jobs = **34.46 ms**	5342.0 ms / 100 jobs = **53.42 ms**	5283.0 ms / 100 jobs = **52.83 ms**
10 jobs	15400.0 ms / 200 jobs = **77.0 ms**	12055.0 ms / 200 jobs = **60.275 ms**	20615.0 ms / 200 jobs = **103.075 ms**	20288.0 ms / 200 jobs = **101.44 ms**
15 jobs	33179.0 ms / 300 jobs = **110.596664 ms**	24439.0 ms / 300 jobs = **81.46333 ms**	43478.0 ms / 300 jobs = **144.92667 ms**	43255.0 ms / 300 jobs = **144.18333 ms**

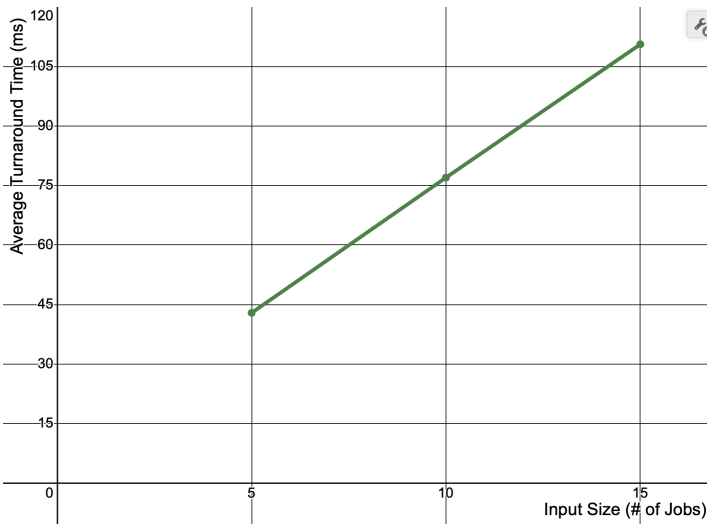
```
Total Average Turnaround Time (20 trials, 5 jobs each trial):
  First Come First Served:      4289.0 ms / 100 jobs = 42.89 ms
  Shortest Job First:           3446.0 ms / 100 jobs = 34.46 ms
  Round Robin (2):              5342.0 ms / 100 jobs = 53.42 ms
  Round Robin (5):              5283.0 ms / 100 jobs = 52.83 ms
```

```
Total Average Turnaround Time (20 trials, 10 jobs each trial):
  First Come First Served:      15400.0 ms / 200 jobs = 77.0 ms
  Shortest Job First:           12055.0 ms / 200 jobs = 60.275 ms
  Round Robin (2):              20615.0 ms / 200 jobs = 103.075 ms
  Round Robin (5):              20288.0 ms / 200 jobs = 101.44 ms
```

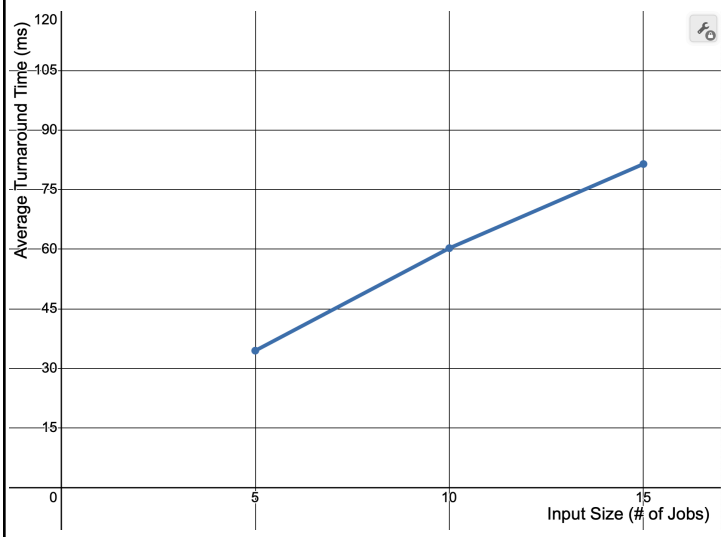
```
Total Average Turnaround Time (20 trials, 15 jobs each trial):
  First Come First Served:      33179.0 ms / 300 jobs = 110.596664 ms
  Shortest Job First:           24439.0 ms / 300 jobs = 81.46333 ms
  Round Robin (2):              43478.0 ms / 300 jobs = 144.92667 ms
  Round Robin (5):              43255.0 ms / 300 jobs = 144.18333 ms
```

b. Plot a graph of each algorithm, average turnaround time vs input size (# of jobs), and summarize the performance of each algorithm based on its own graph.

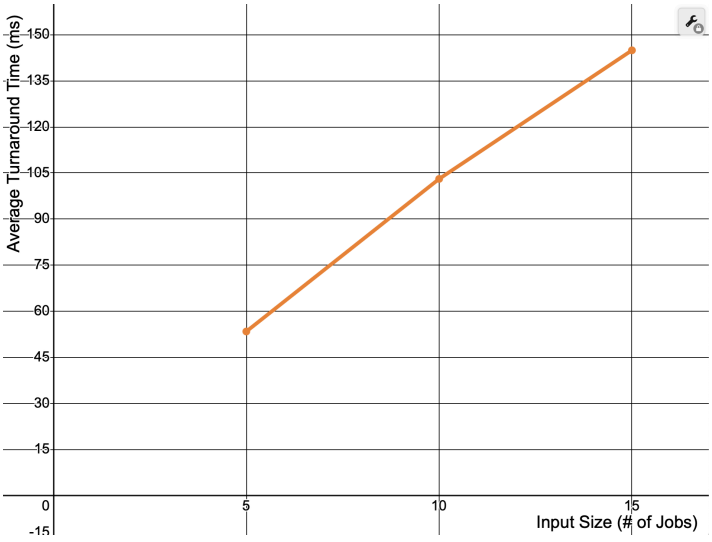
First Come First Served:



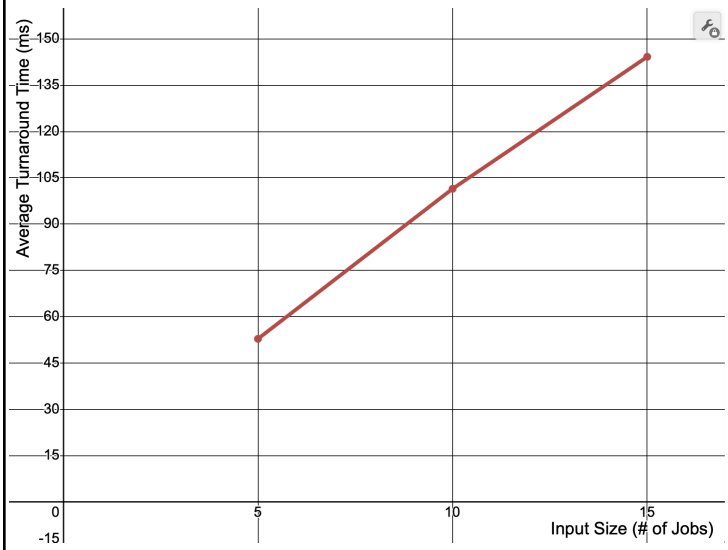
Shortest Job First:



Round Robin (2):



Round Robin (5):



First Come First Served:

The performance of the FCFS algorithm is relatively linear in this example, as there is about the same increase in average turnaround time going from an input size of 5 to 10 as there is going from input size 10 to 15 (~34 ms per 5 jobs). However, this is subject to change as the lengths of the jobs are randomly generated. The FCFS algorithm is more efficient than both of the Round Robin algorithms for all three input sizes, but not as efficient as the Shortest Job First algorithm.

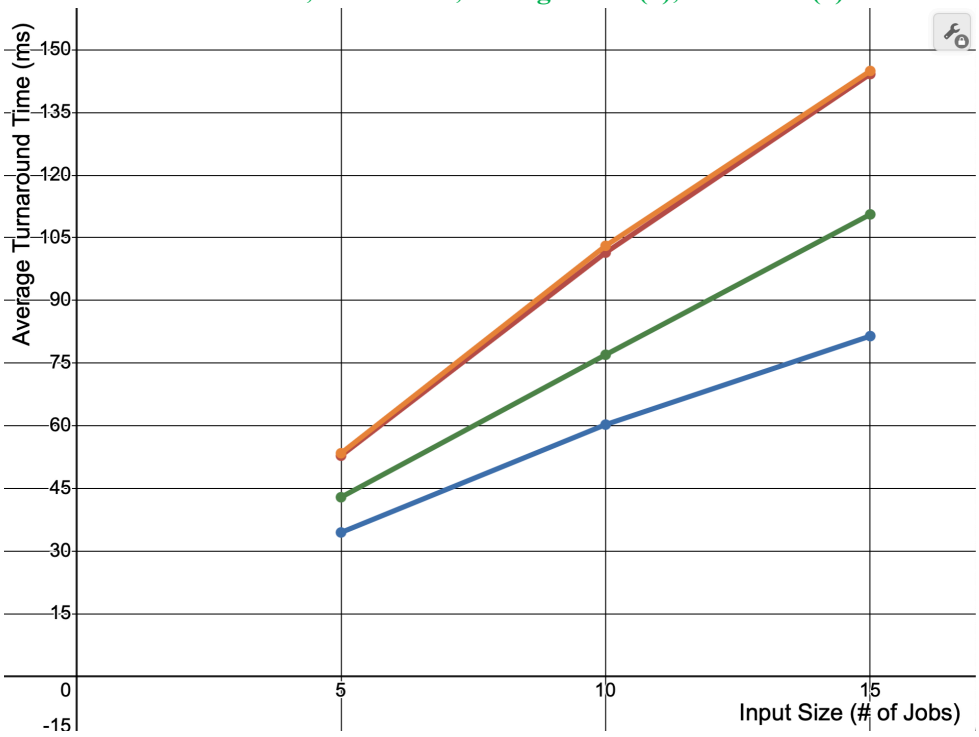
Shortest Job First:
The Shortest Job First algorithm is the most efficient of the four algorithms, as it has the lowest average turnaround time for all three size inputs. In this example, the SJF algorithm increases more between input sizes 5 and 10 than between input sizes 10 and 15, although this is subject to change as the values are randomly generated.

Round Robin (2):
The Round Robin algorithm with time slice 2 is not particularly efficient, especially compared to the FCFS and SJF algorithms. In this example, the average turnaround time increases more between input sizes 5 and 10 than between input sizes 10 and 15, though this is subject to change depending on the randomly generated values. This algorithm is likely to become less efficient with a greater input size and/or greater length of jobs, seeing as it will take more iterations through the entire array to complete all the jobs (especially with a smaller time slice).

Round Robin (5):
The Round Robin algorithm with time slice 5 has a similar level of efficiency as RR with time slice 2, though the time slice of 5 is slightly more efficient than the time slice of 2, especially with greater input sizes and/or jobs with greater lengths. This algorithm is not as efficient as the FCFS or SJF algorithms. In this particular example, the RR algorithm with time slice 5 is slightly more linear than RR with time slice 2, though this is subject to change depending on the values. This algorithm is also likely to become less efficient with a large number of jobs and/or greater job lengths, though not as much as RR with a time slice 2 as a greater portion of each job can be completed during each iteration.

c. Plot all four graphs on the same graph and compare the performance of all four algorithms. Rank four scheduling algorithms. Try giving the reasons for the findings.

All Four: Green = FCFS, Blue = SJF, Orange = RR(2), Red = RR(5)



Results:
The Shortest Job First algorithm had the lowest average turnaround time for each input size (5, 10, and 15), and was therefore the most efficient. The First Come First Serve algorithm was the next most efficient algorithm, and the least efficient algorithms were the Round Robin algorithms. The two Round Robin algorithms had relatively similar results, though RR with time slice 5 performed slightly better than RR with time slice 2, especially with input size 10. The gap in efficiency between the SJF algorithm and the other three algorithms increased as the input size increased, seeing as the average turnaround time increased at a greater rate for FCFS and the two RR algorithms than for SJF. Similarly, the gap in efficiency between FCFS and the two RR algorithms also increased as input size increased, as the average turnaround time increased at a greater rate for the two RR algorithms than for FCFS.

Efficiency (Most → Least):
SJF, FCFS, RR(5), RR(2)

d. Conclude your report with the strength and constraints of your work. At least 100 words. (Note: It is reflection of this project. If you have a chance to re-do this project again, what you like to keep and what you like to do differently in order get a better quality of results.)

One strength of my code is ample options for input sizes and types. Options for input include randomly generated job sizes or input from a user file, the choice of either a single test with formatted output or a 20 trial test that averages the turnaround times for all trials, and the choice of either 5, 10 or 15 job inputs. Other strengths include correctly performing all calculations and all algorithms being in one file for easier testing.

Some constraints include limitations on input size and job length. Input size is limited to either 5, 10, or 15 jobs for all randomly generated jobs, though input from a user file is more open to allow for more than 15 jobs. Job length is also limited, as possible job lengths are between 2 and 25 inclusive.

If I had a chance to re-do this project, some changes I would make include having all four algorithms in different files, cleaning up output, and splitting up the code into helper methods more efficiently. I originally wanted to have each algorithm in a separate file and have each algorithm call the others, however I was worried that since the files were not zipped, there could be some issues with calling methods in other files. I also had some issues with formatting output, as some jobs with longer names (e.g. Job14) or longer times resulted in output tables being skewed. I would like to clean this up and fix the formatting of the tables for larger tests in the future. In terms of splitting up code, combining all four algorithms into a single file and allowing for more user options (input size, single or 20 trial test, and random generation or user file) made it so that the main function is pretty large. I would like to split up main into multiple helper functions, such as a separate function for generating the input file, asking users questions, etc.